Chapter 5 - File attributes and NTFS permissions

We've already mentioned both file attributes and NTFS permissions throughout this book. In this chapter we'll take a closer look at file attributes, and discuss how to view and change attributes from the command line. We'll also examine NTFS permissions, which are considerably more powerful than file attributes, and discuss how to view and alter them as well.

WHAT ARE FILE ATTRIBUTES?

"File attributes" are basically pieces of metadata that contain additional information about the file. In Windows, files generally have their names, their types (defined by the file's extension), and their timestamps. (Certain kinds of files, such as MP3 music files, have additional kinds of metadata, such as the album and artist name.) With file attributes, however, there are four additional pieces of information that you can add to a file.

These four pieces of information are:

-Archive, represented with the A character. The Archive attribute indicates whether or not a file needs to be backed up – basically, whether or not it has been backed up since the last time the file was altered. The Windows Backup feature makes heavy use of the Archive attribute to determine if a file needs to be backed up or not.

-Hidden, represented with the H character. Hidden files are made invisible to the user – they do not show up in the Windows Explorer window, and are not listed

when you use the DIR command. However, we've already explained how to view hidden files with the DIR /A command. Files are generally hidden to protect them from casual deletion – when editing a Microsoft Word document, for example, the temporary files that Word generates are hidden.

-Read-only, represented by the R character. When a file has the Read-only attribute, it cannot be altered. You can open it up and view it, or run it (if it's a program), but you cannot make any changes to do it. Nor can you delete it – if you attempt to delete a file protected by the Read-only attribute, you will receive the "Access is denied" error message.

-System, represented by the S character. Windows uses the System attribute to mark which files are part of the Windows operating system. Needless to say, deleting files marked with the System attribute is a bad idea, since it can result in your computer malfunctioning or even rendering Windows unbootable. For that reason, the System attribute is almost always combined with the Read-only attribute.

LISTING FILE ATTRIBUTES

How do you find out what attributes a file possesses? You can do that with the ATTRIB command. Enter ATTRIB at the command prompt, and ATTRIB will list every file (including the hidden ones) in the current directory, along with their attributes. The output will look something like this:

C:\>attrib

A C:\autoexec.bat

A SHR C:\bootmgr

A SHR C:\BOOTSECT.BAK

A C:\config.sys

A SH I C:\hiberfil.sys

A SH C:\pagefile.sys

(Note that ATTRIB doesn't generally work with absolute file paths – for most applications of the command, you will have to use it in the current directory.)

As you can see, this is a listing of the root directory of a C drive, and the files' attributes are listed on the left side of the screen, with A for Archive, S for System, H for Hidden, and R for Read-only.

(The I attribute for the C:\hiberfil.sys attribute is a rare one – it simply means a file should not be listed by Windows's built-in indexing for file search. Most regular users, and a good many administrators, will have no need for this attribute.)

CHANGING FILE ATTRIBUTES

You can also use the ATTRIB command to change a file's attributes. For instance, if you wanted to mark the test.txt file as Read-only, you would use this command:

ATTRIB +R test.txt

And if you later changed your mind and wanted to remove the Read-only attribute, you would type this command:

ATTRIB –R test.txt

You can also change more than one attribute at the same time. Let's say you wanted to mark test.txt both as a Hidden file and as a Read-only file. Rather than typing the ATTRIB command twice, you can change both attributes at once:

ATTRIB +R +H test.txt.

And you can also remove both attributes simulatenously:

ATTRIB –R –H test.txt

Using the plus sign (+), the minus sign (-), and the one-letter codes for the four attributes, you can change the attributes on files at your leisure.

THE SIX NTFS PERMISSIONS

While file attributes are useful, they are obsolete – they're a holdover from the old days of DOS. NTFS file and directory permissions allow you to exercise more powerful and more granular control over individual files. In essence, NTFS permissions allow you to mark a file as Read-only, but only for certain users – other users might have permission to modify and even delete the file. And this is only the beginning of what you can do with NTFS permissions.

First, though, what is NTFS? NTFS stands for "New Technology File System." (A "file system" is a method for organizing the information on a hard disk so the operating system and applications can find it easily.) Microsoft began working on NTFS in the early 1990s as a replacement for the older File Allocation Table (FAT) file system. NTFS has numerous advantages over FAT – it supports larger disk sizes, larger file sizes, is generally faster, and permits you to use NTFS permissions.

But what are NTFS permissions? NTFS permissions are a way of granting specific users specific permissions to files. Let's say that you have two users on a system – Caina Amalas (with a username of camalas) and Lucan Maraeus (with a username of lmaraeus). If you have a file or a directory, you can set different permissions for the two users. You could give camalas full control over the file, while giving lmaraeus permission to only read the file.

There are six major NTFS permissions. Each of the "big six" are composed of many smaller permissions, but for the purposes of most users and administrators, the six major permissions are enough. They are:

-List Folder Contents. This lets a user see the files in a specific directory, but not necessarily open, run, or modify them. To put it simply, a user can use the DIR command to list the files in a directory, but can do nothing with them.

-Read. This lets a user view the contents of a file. Note that if it's a program file, the user cannot run it – he needs the Read & Execute permission for that. Needless to say, the user will not be able to delete and modify any files.

-Read & Execute. This gives the user permission to run an executable file, but not permission to modify or delete it.

-Write. This lets a user both read and make changes to a file. However, the user still cannot delete the file. (Of course, a user could erase all the text or data in the file, leaving the file blank, but he cannot delete the file entirely.) If a directory has the write permission, it allows the user to add files to it.

-Modify. The modify permission allows a user to read, make changes to, run as a program, and delete a file. Basically, the user can do everything to file except change its permissions or change its ownership.

-Full control. The user can do any action to a file – deleting it, modifying it, and changing its ownership and permissions.

You'll notice that the higher permissions are a superset of the preceding permissions. Modify is a combination of Write, Read & Execute, and Read, while Full Control is a combination of the previous five permissions. The full list of the permissions for a file or a directory is called its ACL – its Access Control List.

LISTING AND CHANGING NTFS PERMISSIONS

Listing and changing a file's or a directory's NTFS permissions from the Command Prompt uses the same command – the ICACLS command. The ICACLS command is extremely complicated, and has several screens worth of options and command switches – you could write an entire short book about ICACLS on its own. However, here we'll discuss how to use ICACLS to do some basic permission changes.

To find out the NTFS permissions of a file named test.txt, use this command:

ICACLS test.txt

The command should produce output that looks something like this:

test.txt NT AUTHORITY\SYSTEM:(I)(F)

BUILTIN\Administrators:(I)(F)

SYSTEMNAME\USERNAME:(I)(F)

The first line shows the name of the file, test.txt. It also shows NT AUTHORITY\SYSTEM, which is one of the groups built into your system. The

I in parentheses after the group name means that the file is inheriting its permissions from a parent folder. (In NTFS permissions, “inheritance” means that you can set the permissions on one directory, and then any subdirectories or files created in that directory “inherit” its permissions. Generally, it’s a good idea never to disable inheritance unless you have a really good idea, since disabled inheritance can cause all kinds of confusing permissions-related errors.) The F in parentheses means that NT AUTHORITY\System has the Full Control permission for the file.

The next two lines show the groups that have access to the file. The BUILTIN\Administrators group likewise has full control. After that, you’ll see a list of users that have access to the file. In this example, SYSTEMNAME\USERNAME is a placeholder for how an entry would look – SYSTEMNAME would be the name of either the local computer or the domain (if the computer is a member of an Active Directory domain) and USERNAME would be the actual username of the user involved.

ICACLS uses one-letter codes to represent the rights as user has to a file.

-N, a user has no access to the file.

-F, a user has full access to the file.

-M, a user has the right to modify the file (this includes the right to delete it).

-RX, a user has the right to read the file and run it as a program, if it’s executable.

-R, a user has only read-only access to the file.

-W, write-only access, a user can make changes to the file, but cannot delete it.

-D, a user has the right to delete the file.

You can use ICACLS to change the permissions on a file. For instance, say you wanted to give the user account camalas full control over the test.txt file:

ICACLS text.txt /grant camalas:(F)

This will grant the camalas user account full control over the test.txt file.

Likewise, if you wanted to deny camalas full control over the file, you would use this command:

ICALCS text.txt /deny camalas:(F)

The camalas user account will then be unable to access the test.txt file at all.

An important note – in NTFS permissions, a Deny permission always overrides

an Allow permission. For instance, if you were grant camalas the Read permission on test.txt, but also deny the camalas account the Full control permission, camalas would be unable to access test.txt in any way. This can be confusing because user accounts can belong to groups, which have their own permission sets, which can conflict with the individual file's permission.

ICACLS is a complicated command, but you should now know the basics of changing permissions from the Command Prompt.

***

Chapter 6 – Wildcards and redirection

We've already mentioned wildcard characters in this book. However, before we move on to Chapter 7, which will discuss commands for moving, copying, deleting, and renaming files, we should look at wildcards in greater detail. A thorough knowledge of the Command Prompt's wildcard characters will let you issue for more effective commands for manipulating files. We'll also look at "redirection", which lets you feed the output from one command into the input for another.

WHAT ARE WILDCARD CHARACTERS?

Wildcards are characters that represent all possible other characters. As you might remember from Chapter 4, the question mark character (?) can represent every other potential character, and the asterisk (*) can represent any potential combination of characters. Why are wildcards useful? Basically, they let your commands affect more than one file at a time.

Let's say you wanted to use the DEL command to erase a file called test.txt:

DEL test.txt

That command (assuming you had the proper permissions) would delete the test.txt file and do nothing else. But if you add wildcards into the mix, the command can affect multiple files. For instance, add a single question mark to the command:

DEL test?.txt

This command will not delete the test.txt file. It will, however, delete every file in the directory that has “test” in its name, “txt”, as its extension, and additional single character before the extension. So test1.txt, test2.txt, and test3.txt, would all be deleted by this single command. Much more efficient than typing DEL over and over again!

However, any of the files that had two characters before the extension – like test10.txt and test11.txt – will not be deleted. This is because the question mark character only serves as a wildcard for a single character. It’s useful if you want your command to affect a very narrow subset of files – if you only want to copy a few files, or to delete a few particular files.

To cast a wider net with your command, you need the asterisk (*) wildcard character. The question mark wildcard character represents a single character, but the asterisk wildcard represents any number of characters. For example, consider what would happen if you used an asterisk instead of a question mark with the DEL command:

DEL test*.txt

This command will delete every single file in the current directory that begins with “test” and has an extension of “txt”. No matter how many characters come after “test” and before “txt”, the DEL command will delete them all.

You can make the DEL command even more powerful by removing the filename and leaving only the asterisk:

DEL *.txt

This command will delete every file in the current directory that has an extension of “txt.” We’ve already mentioned how an asterisk wildcard character can represent any number of characters – here will delete any file, regardless of its name, that has the “txt” extension.

So think about what would happen if you typed this command:

DEL *.*

The use of the asterisk wildcard characters here means that the DEL command will delete any file with any name and any extension. To put it more simply, DEL will delete every single file in the current directory! Needless to say, you should exercise extreme caution while using wildcards with the DEL command.

The wildcard characters work with file manipulation commands other than DEL. You can use the wildcards to make copying and moving files from the Command Prompt far quicker and more efficient.

REDIRECTION

To understand redirection at the Command Prompt, you first to need to understand the ideas of input and output. Input is any information you enter into the computer – a mouse click, or the commands you type at the Command Prompt. Output is information that the computer returns to you – the menu that appears when you right-click the mouse, or the text Command Prompt displays when you enter a command.

Using redirection, you can redirect the output from one command to another command. The most common use of redirection is with the MORE command. If you type the DIR command in a directory that contains a great deal of files, the output will probably scroll off the top of the screen. You could use the DIR /P command to view the output one screen at a time. You have another option with the MORE command and a redirection pipe:

DIR | MORE

The pipe character (|) redirects the output from the DIR command to MORE. The MORE command takes the output and displays it one screen at a time, letting you scroll down line by line. You can also use the pipe character and MORE with the TYPE command. The TYPE command displays the contents of a plain-text file (generally a file with the “txt” extension, though some system INI files are plain-text) on the screen. See this example:

TYPE test.txt | MORE

If test.txt is too long to display on the screen, the MORE command will parse it out one screen of text at a time.

You can also use output redirection create files with the output from the commands. For example, what if you wanted to keep a listing of all the files in a particular directory? You could type the DIR command, of course, but the output would disappear as soon as you closed the Command Prompt window, or even if you typed enough other commands. However, you can avoid the problem if you use DIR with the greater than (>) sign:

DIR > output.txt

When you hit the Enter key, it might seem initially that nothing has happened. However, a new file named "output.txt" has been created in your current directory, and it contains the output from the DIR command. You can display the file with this command:

TYPE output.txt

You can then view the contents of the file.

It is important to take care while using the greater than sign to redirect output. In the previous example, if there's already a file named "output.txt" in the directory, the command will overwrite it, and any data in the original file will be lost. So make sure that you pick a new filename for your output.

However, by using a double greater than sign (>>) you can append the output to an existing file. Rather than overwriting an existing file, the double greater than sign will simply add the output to the end of the existing file. Here's an example:

DIR >> output.txt

You can then read output.txt at your leisure, whether with the TYPE command or with the Notepad application.

Redirecting output to text files might not seem very useful with the DIR command, but it really comes in handy when working with Command Prompt's networking commands. (We'll discuss networking more in Chapter 8.) Many of the networking commands produce long and complicated outputs, and you might want to put the output in a text file to peruse at your leisure.

***

Chapter 7 - Copying, moving, renaming, and deleting files.

In the previous chapters, we've discussed the basics of the Command Prompt, how to use NTFS permissions, and the basics of managing directories. All the directories in the world won't do you any good if you have nothing to store in them, though. In this chapter we will discuss the basics of managing your files from the command line.

COPYING FILES

One of the most useful features of the modern PC is the ability to swiftly copy files from one location to another. Practically every computing device has the ability to copy files. The Windows operating system and the Command Prompt are no different. The basic command for copying files from the Command Prompt is the COPY command. If you wanted to use COPY to copy a file called test.txt to the root directory of a USB flash drive assigned the letter J:

COPY test.txt J:

The command will create a copy of test.txt in the root directory of the J drive.

Like most Command Prompt commands, COPY assumes that you will be copying files from the working directory, and looks there first to find the filenames you specify. However, you can also use COPY with absolute file paths. If test.txt were in the Users directory of the C drive, you would use this

command to copy it to J:

COPY C:\Users\test.txt J:

But what happens if there's already a file named "test.txt" in the root directory of J? If there's already a file there, COPY will pause for your confirmation before continuing:

Overwrite J:\test.txt? (Yes/No/All):

If you hit Y to continue, COPY will overwrite the file. Hit N, and COPY will leave the file intact. (The All option is used when copying multiple files at once - hit A, and COPY will not ask you again if you want to overwrite the files for the remainder of the copying job.)

You can override this prompt with the /Y switch:

COPY C:\Users\test.txt J:\ /Y

With the /Y switch, COPY will overwrite any files without first prompting you to continue. Needless to say, you should be careful using the /Y switch, since you might accidentally overwrite valuable data.

Like most other file manipulation commands, COPY works with the wildcard characters. If you wished to copy every file with the "txt" extension in the Users

directory to J:\, you would use this command:

COPY C:\Users*.txt J:

The COPY command is easy to use, but it is fairly limited, and has only a few available options. For a more powerful copying utility, Command Prompt offers the XCOPY utility. On the surface, XCOPY works just like COPY - you can copy individual files, use wildcards, and so forth. XCOPY's main advantage over COPY is that it allows you to copy subdirectories. In other words, you can copy entire branches of the directory tree from one location to another with the XCOPY command. In this example, XCOPY will copy the entire contents of your C:\Users directory, including all subdirectories, to the USB drive at J:

XCOPY C:\Users*.* J:\ /S /E

You will notice the use of two switches, /S and /E. The /S switch tells XCOPY to copy all the subdirectories in the C:\Users directory. The /E switch tells XCOPY to copy the subdirectories even if they are empty.

Here are a few more useful switches XCOPY offers:

-/M copies only files with the Archive attribute enabled, and then switches it off. This essentially lets you use XCOPY as a crude backup utility.

-/D:m-d-y only copies files changed after the specified date (for instance, for December 27th, 2010, the command would look like this: XCOPY *.* /D:12-27-

2010).

-/C tells XCOPY to continue the copy operation even if there are errors. This is useful when copying large numbers of files from a failing or damaged disk - if you're trying to copy off, say, 85 gigabytes worth of music files, it is quite irritating to have the operation fail two gigabytes into the job due to an error.

-/H tells XCOPY to copy files with the System and Hidden attributes, as well.

-/R overwrites any files with the Read-only attribute in the destination directory that share same name as the copied files. You should be careful with this option, since it can cause data loss.

-/N copies the files using their 8.3 filenames.

-/Y works the same as the /Y switch with the COPY command, and tells XCOPY to overwrite any files of the same name without prompting you first.

The COPY command is good enough for quickly copying small groups of files, but for heavy-duty file copying, you'll want to use the XCOPY command.

MOVING FILES

There are two ways to move a file. You could copy the file to its new destination, and then delete the original source file. This involves unnecessary work. Using